PROJECT DETAIL REPORT

EduMind
Multi-role EdTech Platform

Architecture, entry points, workflow, API catalog, data design, features, security, and runtime guide.

3 **React portals**	**31** **/api endpoints**	**3** **MongoDB models**	**5** **external services**

Scope and method

This report is based on a static review of the repository at C:\edtech-platform, including server routes, middleware, models, React application entry modules, page-level API calls, package manifests, tests, and supplied interface screenshots. It describes what the code implements; it does not claim a live production audit.

Prepared 07 September 2026

1. Executive summary

EduMind is a monorepo that exposes three role-specific React/Vite experiences - Student, Teacher, and Admin - against one Express REST API and shared MongoDB data store. The platform is organized around course creation, catalog discovery, enrollment/payment, learning progress, certification, instructor reporting, and administrative governance.

Student portal :5173	Teacher portal :5174	Admin portal :5175	Express API :5000	MongoDB

What is implemented

- Role-aware account registration, login, JWT recovery on refresh, protected client routes, and server-side role checks.
- Teacher course authoring with lessons, video upload to Cloudinary, edit/delete ownership checks, analytics, and student rosters.
- Student discovery, public course preview, demo or Stripe checkout, enrolled-only course player, progress persistence, lesson completion, certificates, and AI tutor support.
- Admin dashboard operations: list users, change roles, delete users, list all courses, and delete courses.
- Platform safeguards including Helmet headers, CORS policy, global/auth rate limits, Zod request validation, MongoDB ID validation, centralized errors, bcrypt password hashing, and JWT verification.

API count

Route group	Endpoints	Purpose
Authentication	3	Register, login, current user
Courses	15	Catalog, authoring, learning, progress, tutor, certificates
Users	8	Profile, learning, certificate verification, stats, admin users
Payments	4	Checkout, order lookup, demo completion, Stripe webhook
Upload	1	Teacher/admin video upload
Total /api	31	Mounted under /api/* in server.js
Plus root	1	GET / returns server status text

2. System architecture

The repository is a practical monorepo: the applications are separated by user role, while the backend owns identity, authorization, business rules, data persistence, and external-service calls. The packages/ directory exists as a placeholder for future shared modules but is not currently the shared runtime boundary.

Runtime layers

Layer	Implemented files / technology	Responsibility
Web portals	apps/student, apps/teacher, apps/admin; React 19, Vite, React Router, Tailwind	Role-specific navigation, pages, local state, guarded routes, API requests
Shared client convention	src/utils/api.js; AuthContext.jsx; ProtectedRoute.jsx in each portal	Axios base URL normalization, Bearer token injection, session recovery, client-side role redirect
API	server/server.js; Express 5	Middleware registration, CORS, limits, JSON parsing, route mounting, error handler
Domain routes	server/routes/*.js	Auth, course, user/admin, payment, upload behavior
Persistence	Mongoose models: User, Course, Order	MongoDB document validation and relationships
External integrations	Cloudinary, Stripe, Gemini API, MongoDB Atlas, Vercel/Render deployment	Video hosting, payments, AI responses, database hosting, deployment

Request path

Browser route	React page	Axios interceptor	Express middleware	Route + model	JSON response

For authenticated requests, the interceptor reads token from localStorage and sends Authorization: Bearer <token>. The auth middleware verifies it, stores the decoded id and role on req.user, and restrictTo(...) then enforces the server-side role boundary where required.

3. Entry points and file responsibilities

There are four runtime entry points. Each frontend entry mounts its own provider hierarchy and router; the server entry builds the shared API application and only opens a database connection/listener outside test mode.

Entry file	Starts	Key work performed
apps/student/src/main.jsx	Student portal on Vite port 5173	Mounts React StrictMode, AuthProvider, ToastProvider, and App
apps/teacher/src/main.jsx	Teacher portal on Vite port 5174	Same provider structure; teacher-specific App routes
apps/admin/src/main.jsx	Admin portal on Vite port 5175	Same provider structure; admin-specific App routes
server/server.js	Express API (default port 5000)	Loads env, applies Helmet/rate limits/CORS/JSON, mounts /api routers, connects Mongoose, attaches error handler

Frontend route surfaces

Portal	Public routes	Protected routes
Student	/, /login, /register, /courses, /course/:id, /about, /contact, /verify-certificate/:certId	/profile, /my-courses, /demo-checkout/:orderId, /learn/:id
Teacher	/, /login, /register, /courses, /course/:id, /about, /contact	/profile, /my-courses, /instructor-dashboard, /create-course, /edit-course/:id
Admin	/, /login, /register, /courses	/profile, /admin-dashboard

Important file map

Area	Files	How they work together
Routing	apps/*/src/App.jsx	Declares browser routes and wraps restricted pages in ProtectedRoute
Session	apps/*/src/context/AuthContext.jsx	On load: reads local token, calls /auth/me, sets authenticated user or clears invalid token
HTTP	apps/*/src/utils/api.js	Normalizes VITE_API_URL and adds Bearer token to every request
Access control	server/middleware/auth.js; restrictTo.js	JWT verification first; allowed role list second
Validation	server/utils/validationSchemas.js; middleware/validate.js	Zod schemas parse/sanitize auth, course, and progress request data
Errors	server/middleware/AppError.js; errorHandler.js	Normalizes operational errors, database errors, and JWT errors

4. Core workflows

A. Account and role workflow

Register / Login	**Zod validation**	**bcrypt compare/hash**	**JWT for 7 days**	**localStorage**	**/auth/me recovery**

The server accepts Student, Teacher, or Admin during registration. The token carries the user id and role. Client-side ProtectedRoute improves navigation, but API endpoints also enforce identity and roles, preventing a role switch in the browser from granting backend access.

B. Teacher authoring workflow

Create course	**Optional video file**	**POST /upload/video**	**Cloudinary URL**	**POST /courses**	**Published/Draft catalog**

A teacher creates a course with title, description, price, duration, lessons, metadata, outcomes, and requirements. Video files are held in Multer memory storage, streamed to Cloudinary as video resources, and the returned secure URL is placed in a lesson payload. Course creation/update/delete checks both role and ownership; Admin has an override.

C. Student purchase and learning workflow

Public preview	**Create checkout**	**Pending order**	**Stripe or demo**	**Enrollment**	**Secure player**

Public course preview deliberately removes lesson videoUrl and content. Checkout creates an Order. With STRIPE_SECRET_KEY configured, the API returns a Stripe Checkout URL; otherwise it returns a local demo checkout route. A Stripe webhook or demo-complete endpoint marks the order completed, adds the course to enrolledCourses, and increments studentsCount. The /courses/:id/learn endpoint returns private lesson material only to enrolled users, the course owner, or an Admin.

D. Learning, completion, and certificate workflow

Open player	**Restore progress**	**Save time/index**	**Complete lesson**	**100% completion**	**Public verification**

The player fetches secure course content and saved progress. It posts current lesson index/time and can mark a specific lesson complete. Once all course lesson ids are present in completedLessons, the server creates a UUID certificate id and HMAC-SHA256 hash, stores it on the user, and exposes a public verification route by certificate id.

5. Data model and relationships

MongoDB is accessed through Mongoose. References link User to Course and Order; embedded schemas hold lesson, progress, and certificate records. The following diagram captures the operational relationships rather than every scalar field.

User	enrolledCourses -> Course	progress[]	completedLessons[]	certificates[]	Order

Core collections

Model	Key fields	Relationships and behavior
User	name, email (unique), password (select:false), role, avatar	enrolledCourses references Course; progress embeds courseId/index/time; certificate embeds courseId/UUID/hash/date
Course	title, description, instructor, lessons, price, duration, status, category, level, rating, studentsCount	instructor references User; lessons are embedded subdocuments holding title, content, videoUrl, duration
Order	user, course, amount, status, paymentMethod, stripeSessionId	References User and Course; transitions from pending to completed after successful payment path

Consistency notes

- Enrollment is idempotent in both manual enroll and payment completion: the code avoids adding an already-enrolled course and avoids a second student count increment.
- Course completion is idempotent: a completed lesson id is not pushed twice, and an existing certificate is reused rather than duplicated.
- A student's stored completedLessons is a single cross-course list. The completion handler derives course completion by intersecting that list with the current course's lesson ids.
- The models use timestamps, supporting enrollment/user/order creation chronology in the UI and reporting routes.

6. REST API catalog (1 of 2)

Counted API surface: 31 routes mounted under /api. The access column reflects server-side enforcement, not only client navigation guards. A route marked Auth requires a valid Bearer JWT.

Method	Path	Access	Behavior
POST	/auth/register	Public	Validate account fields, hash password, create user, return JWT and sanitized user
POST	/auth/login	Public	Validate credentials, compare bcrypt hash, return JWT and sanitized user
GET	/auth/me	Auth	Return current user without password
POST	/courses	Teacher/Admin	Create course owned by caller
GET	/courses	Public	Return catalog courses with instructor name
GET	/courses/:id	Public	Return course preview; strips lesson content and videoUrl
GET	/courses/:id/learn	Auth + enrollment/owner/admin	Return full private course including lesson content and video URLs
PUT	/courses/:id	Teacher/Admin + owner/admin	Validate and update a course
DELETE	/courses/:id	Teacher/Admin + owner/admin	Delete a course
POST	/courses/:id/enroll	Auth	Add user enrollment and increment studentsCount
GET	/courses/instructor/my-courses	Teacher/Admin	Return courses owned by caller
GET	/courses/instructor/analytics	Teacher/Admin	Return course/student aggregate counts and per-course statistics
GET	/courses/instructor/courses/:id/roster	Teacher/Admin + owner/admin	Return enrolled students and computed lesson-completion percentage
POST	/courses/:id/progress	Auth + enrollment/owner/admin	Store lesson index and playback time
GET	/courses/:id/progress	Auth + enrollment/owner/admin	Return stored index/time or defaults

7. REST API catalog (2 of 2)

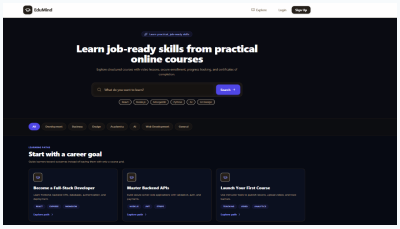
Method	Path	Access	Behavior
POST	/courses/:id/lessons/:lessonIndex/ai-tutor	Auth + enrollment/owner/admin	Use Gemini if configured; otherwise return context-aware mock tutor response
POST	/courses/:id/lessons/:lessonId/complete	Auth + enrollment	Mark lesson complete; issue certificate at full course completion
GET	/courses/:id/certificate	Auth + enrollment	Return caller's certificate for course
GET	/users/public-stats	Public	Return platform totals and most-followed published course
GET	/users/verify-certificate/:certId	Public	Return certificate/student/course verification details
GET	/users/my-learning	Auth	Return populated enrolled courses
GET	/users/profile	Auth	Return populated profile, enrollments, and certificates
PUT	/users/profile	Auth	Update name and/or avatar; return refreshed profile
GET	/users	Admin	List sanitized users, newest first
PUT	/users/:id/role	Admin	Change role to Student, Teacher, or Admin
DELETE	/users/:id	Admin	Delete a user
POST	/payments/create-checkout-session	Auth	Create Stripe or demo pending order and return redirect URL
GET	/payments/orders/:orderId	Auth + order owner	Return caller's populated order
POST	/payments/demo-complete	Auth + order owner	Complete demo order and enroll caller
POST	/payments/webhook	Stripe signature	Process checkout.session.completed; complete order and enroll user
POST	/upload/video	Teacher/Admin	Accept multipart video, stream to Cloudinary, return secure URL

Server root: GET / returns a simple running-status string and is not counted in the 31 /api routes. Route prefixes are set in server/server.js: /api/auth, /api/courses, /api/users, /api/upload, and /api/payments.

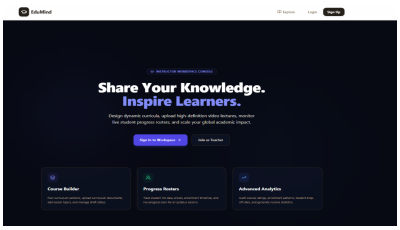
8. Feature coverage by role

Role	Primary features	Main supporting APIs
Student	Browse catalog; preview course; login/register; purchase via Stripe/demo; my learning; video player; progress; completion; AI tutor; profile; certificate verification	auth, courses public/learn/progress/completion/certificate, users profile/learning/verify, payments
Teacher	Create/edit/delete owned courses; upload lesson videos; manage publication metadata; view portfolio courses; dashboard analytics; roster and completion percentages	courses create/update/delete/instructor routes, upload/video
Admin	Role-gated dashboard; list and manage users; update roles; remove users; review and remove catalog courses; profile	users admin routes, courses catalog/delete

Included interface evidence



Student portal
Supplied repository screenshot



Teacher portal
Supplied repository screenshot

Admin portal
Supplied repository screenshot

9. Security, resilience, and integrations

Implemented controls

Control	Where	Effect
HTTP hardening	Helmet in server.js	Adds common security headers
Rate limiting	server.js	300 requests/IP/15 min globally; 50 auth attempts/IP/15 min
CORS	server.js	Allows configured origins in production and local Vite ports in development; Vercel pattern accepted
Authentication	auth.js	Requires correctly formed Bearer token; jwt.verify checks integrity and expiry
Authorization	restrictTo.js plus ownership checks	Limits course management/upload and admin operations by role; protects course ownership
Input validation	Zod schemas + validate middleware	Validates auth, course, progress payloads and MongoDB ids before work
Password protection	auth route + User schema	bcrypt hash; password excluded by default and removed from auth responses
Content gating	courses route	Public preview strips private lesson content/video; learn route checks enrollment
Error handling	errorHandler.js	Maps operational, Mongoose, duplicate-key, invalid/expired JWT errors to consistent responses

External services and configuration

Service	Purpose	Configuration
MongoDB / Atlas	User, course, and order data	MONGO_URI or MONGODB_URL
Cloudinary	Video upload and secure media URL	CLOUD_NAME, CLOUD_API_KEY, CLOUD_API_SECRET
Stripe	Hosted checkout and signed webhook	STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET
Gemini	Lesson-context AI tutor response	GEMINI_API_KEY; mock fallback when absent
Vercel / Render	Portal and API deployment references	VITE_API_URL and server CORS origins

10. Development, testing, and deployment

Local startup

Component	Working directory	Command	Default endpoint
Backend	server	npm run dev	http://localhost:5000/api
Student	apps/student	npm run dev	http://localhost:5173
Teacher	apps/teacher	npm run dev	http://localhost:5174
Admin	apps/admin	npm run dev	http://localhost:5175

Each frontend expects VITE_API_URL to point to the shared API base. Its local api.js adds /api only if the environment value does not already end in /api. In the server, NODE_ENV=test prevents the normal database listener from starting, allowing Supertest to import the Express application.

Test suite

The server uses Vitest, Supertest, and mongodb-memory-server. The reviewed tests exercise successful and invalid registration/login/current-user calls; teacher vs. student course creation; validation and ownership checks; enrollment idempotency; progress bounds; lesson completion; certificate creation/retrieval; and unauthorized paths. The README describes a 23-test integration suite.

Operational flow for deployed payment

Client requests checkout	**API records pending Order**	**Stripe hosted payment**	**Stripe webhook signs event**	**API completes order**	**User enrolled**

The demo mode retains the same business outcome without Stripe: an order is created, the student sees the local demo checkout page, and /payments/demo-complete finalizes the order/enrollment. This makes the platform demonstrable without payment credentials.

11. Implementation notes and source guide

The codebase is intentionally organized around a central backend and three focused client experiences. The strongest boundaries are server-side: JWT verification, role restrictions, course ownership checks, enrollment checks, validation, and data-model relationships. The frontend repeats certain foundational files across portals (API helper, auth context, guard, navigation) rather than importing them from packages/, which is a clear future consolidation opportunity but does not change the current runtime behavior.

Reference files reviewed

Concern	Primary files
Server composition	server/server.js
Routes	server/routes/auth.js, courses.js, users.js, payments.js, upload.js
Data	server/models/User.js, Course.js, Order.js
Security and validation	server/middleware/auth.js, restrictTo.js, validate.js, errorHandler.js; server/utils/validationSchemas.js
Student portal	apps/student/src/main.jsx, App.jsx, context/AuthContext.jsx, utils/api.js, pages/*
Teacher portal	apps/teacher/src/main.jsx, App.jsx, context/AuthContext.jsx, utils/api.js, pages/*
Admin portal	apps/admin/src/main.jsx, App.jsx, context/AuthContext.jsx, utils/api.js, pages/*
Tests and configuration	server/tests/*.test.js, server/tests/testSetup.js, package.json files, Vite configs, README.md

Report conclusion

EduMind implements a complete role-based learning lifecycle: users discover courses, instructors publish them, students purchase/enroll and learn securely, completion generates verifiable credentials, and administrators govern users and course catalog data. The platform currently exposes 31 versioned API endpoints and maintains one additional root status endpoint.

End of report